Protocol Sensitivity in Unsupervised Industrial Anomaly Detection: Threshold Transfer, Retraining Risk, and Inference Cost

Sarjan Shah[1] and Abhi Shah[1]

[1]*Department of Information Technology, Lok Jagruti Kendra University, Ahmedabad, Gujarat, India*

shahsarjan968@gmail.com
abhi.shah@ljku.edu.in

ABSTRACT

Comparisons of unsupervised anomaly detectors for industrial sensor data usually report a winning method, yet the winner can depend on evaluation choices that are rarely varied. This paper measures how the definition of the normal training period, the threshold-calibration rule, the sensor set and the selection of retraining data change the conclusions. Nine lightweight detectors and a small LSTM autoencoder are evaluated with threshold-free and threshold-dependent metrics, event recall, detection delay and block-bootstrap confidence intervals on four benchmarks: SKAB, the official SWaT and WADI releases, and HAI. Ranking quality alone rarely separated the detectors: on SWaT all ten reached AUROC 0.81–0.86 with overlapping intervals, whereas on HAI PCA reconstruction and LOF led (0.93, 0.92). Protocol choices moved MCC by up to 0.8 for the same detector, and a widely shared re-upload of SWaT, which stores the attack period's normal rows in its normal-operation file, reordered the detectors. Thresholds set only from normal data failed wherever chemical analysers drifted between the normal and the attack recordings: on SWaT the *z*-score, One-Class SVM, LOF, PCA reconstruction and the LSTM autoencoder raised alarms on 42–75% of normal readings, whereas COPOD, ECOD, HBOS and Isolation Forest stayed below 12%; one analyser missing from the re-upload raised the *z*-score's false-alarm rate from 1.4% to 42%. In a single-stream retraining simulation, Isolation Forest recovered only when the retraining data were known to be normal and collapsed or absorbed attacks under label-free selection, whereas Elliptic Envelope stayed stable; a drift trigger saved refits but its checks cost more than they saved. Median single-reading latency in a one-CPU, 1 GB container ranged from microseconds to about 740 ms. For condition-monitoring systems, the results argue for choosing detectors by threshold stability and retraining needs, and for reporting protocols in enough detail to tell what a benchmark number measures.

Keywords: unsupervised anomaly detection; multivariate time series; evaluation protocol; concept drift; retraining policy; resource-constrained inference

Sarjan Shah et al. This is an open-access article distributed under the terms of the Creative Commons Attribution 3.0 United States License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

1. INTRODUCTION

Pumps, valves and treatment plants are instrumented with sensors whose joint behaviour changes before or during a fault. Because faults and attacks are rare and costly to label, condition monitoring often relies on *unsupervised* (novelty) detectors that are fitted to normal operation and flag departures from it (Chandola et al., 2009; Mukherjee, 2021). In practice, false alarms decide whether such a monitor is trusted, which is why deployed systems add contextual filtering and adaptive thresholds on top of the detector (Lee, 2017). For an engineer choosing such a detector for a resource-limited monitoring node, three practical questions matter: which detector separates anomalies from normal operation best, whether that answer survives the passage of time once the detector is deployed, and what the detector costs to run. In prognostics and health management (PHM), such a detector is the first stage of the monitoring chain: its alarms start diagnosis and maintenance decisions, so a threshold that silently stops transferring, or a retraining step that learns a fault as normal, lowers the dependability of everything downstream.

Published comparisons give less guidance on these questions than their headline numbers suggest. Large benchmark studies report that no family of methods is consistently best and that deep models often do not beat well-chosen classical ones (Audibert et al., 2022; Schmidl et al., 2022; Sarfraz et al., 2024). A parallel line of work shows that evaluation choices, such as point adjustment, metrics that reward trivial detectors, and flawed benchmark labels, can manufacture apparent progress (Kim et al., 2022; Garg et al., 2021; Wu & Keogh, 2023; Liu & Paparrizos, 2024). These critiques concern mainly metrics and labels. Less attention has been paid to the other protocol choices an applied study must make: how the training and test data are defined, how a detection threshold is calibrated, and, once deployed, how and on which data a detector is retrained.

This study began as a conventional detector comparison on the Skoltech Anomaly Benchmark (SKAB) (Katser & Kozitsin, 2020) and the Secure Water Treatment testbed (SWaT) (Goh et al., 2016; Mathur & Tippenhauer, 2016). An initial analysis, built with choices that are common in applied work, named clear winners on each dataset and a large benefit from drift-triggered retraining. A subsequent audit showed that those conclusions rested on three protocol choices: the SWaT copy used (a public re-upload) stores the normal-labelled rows of the attack period in the file named for normal operation, so the model had been trained on the normal intervals *between* attacks rather than on the separate normal run; thresholds had been calibrated with different rules for different detectors; and retraining data had been selected with ground-truth labels. Instead of only correcting these choices, this paper measures how much each one changes the conclusions, re-runs SWaT on the official release, and adds two further benchmarks (HAI and WADI) to test whether the corrected findings hold.

The contributions are empirical and methodological, not algorithmic:

1. A controlled comparison of nine lightweight unsupervised detectors and a small LSTM autoencoder on SKAB, SWaT, WADI and HAI under standard, documented protocols, reported with threshold-free metrics (AUROC, AUPRC), threshold-dependent metrics (MCC, event recall, false-alarm rate) and block-bootstrap confidence intervals (Section 5).
2. A protocol-sensitivity analysis showing how the split definition, the training protocol, the threshold rule and the sensor set change the measured performance and the ranking of detectors, including a check of a public SWaT re-upload against the official release (Section 5.5).
3. A retraining-policy study on SWaT's genuine normal run and attack period that compares never, periodic, drift-triggered and every-batch retraining, with retraining data selected with or without labels (Section 6).
4. Single-reading latency, throughput, model size and peak memory for every detector inside a one-CPU, 1 GB ARM64 container (Section 7).

The study answers four research questions:

RQ1
Under standard, documented protocols with uncertainty estimates, do lightweight detectors differ in how well they rank anomalies on SKAB, SWaT, WADI and HAI?

RQ2
How much do the split definition, the training protocol and the threshold rule change measured performance and the ranking of detectors?

RQ3
Does drift-triggered retraining outperform periodic or every-batch retraining, and what happens when retraining data are selected without labels?

RQ4
What does each detector cost per reading under a one-CPU, 1 GB memory cap?

2. RELATED WORK

Detectors.

Classical unsupervised detectors differ in what they treat as normal. Distance- and density-based methods such as the Local Outlier Factor (LOF) score a point by the relative density of its neighbourhood (Breunig et al., 2000); boundary methods such as the One-Class SVM enclose the bulk of the training data (Schölkopf et al., 2001); covariance methods flag large Mahalanobis distances, with the Minimum Covariance Determinant (MCD) giving a robust estimate (Rousseeuw & Van Driessen, 1999); PCA reconstruction scores the residual outside a low-rank normal subspace (Shyu et al., 2003); Isolation Forest isolates points by random partitioning (Liu et al., 2008); and the histogram- and copula-based HBOS, COPOD and ECOD combine per-feature tail probabilities and are fast and nearly parameter-free (Goldstein & Uchida, 2016; Li et al., 2023; Zhao et al., 2019). Deep sequence models, including LSTM encoder-decoders (Malhotra et al., 2016), prediction-error detectors (Hundman et al., 2018), adversarial autoencoders (Audibert et al., 2020), graph networks (Deng & Hooi, 2021) and transformers (Tuli et al., 2022), report strong results on SWaT, usually with protocols and metrics that differ from paper to paper.

What benchmark studies agree on.

Comparisons that put conventional, machine-learning and deep detectors side by side find that no family wins consistently (Audibert et al., 2022), that multivariate problems are harder on average (Schmidl et al., 2022), and that simple or linear baselines match many recent deep models once inflated metrics are removed (Sarfraz et al., 2024). Tabular benchmarks reach similar conclusions (Goldstein & Uchida, 2016; Han et al., 2022). A result of the form "detector X wins on dataset A but not on B" is therefore expected and is not, on its own, a contribution.

Where evaluations go wrong.

Point adjustment credits a whole event when any of its points is flagged and can make random scores look competitive (Kim et al., 2022). Point-wise F1 and accuracy can reward detectors that flag almost everything, which the Matthews correlation coefficient (MCC) does not (Chicco & Jurman, 2020; Garg et al., 2021). Threshold-dependent metrics make comparisons hinge on the threshold rule; threshold-free AUC measures and their range-aware extensions avoid this (Paparrizos et al., 2022; Liu & Paparrizos, 2024), and range-based precision and recall account for events rather than points (Tatbul et al., 2018). Benchmark data can itself be flawed (Wu & Keogh, 2023), and the PHM community has called for shared measurement and evaluation practice for health-monitoring methods (Weiss & Brundage, 2021). These studies examine metrics and labels; the protocol choices examined here (split semantics, threshold calibration rule, and the label assumption in retraining) are complementary.

Drift and retraining.

Concept-drift research distinguishes detecting a change, understanding it and adapting to it (Gama et al., 2014; Lu et al., 2019). Two-sample tests on feature distributions are a simple and competitive way to detect shift (Rabanser et al., 2019); adaptive windows such as ADWIN resize the reference automatically (Bifet & Gavaldà, 2007); streaming anomaly benchmarks such as NAB score detectors online (Ahmad et al., 2017). For an anomaly detector, adapting to drift carries a specific risk: data used for retraining may contain the very anomalies the detector should flag. Retraining policies have been compared directly for anomaly detectors in IT operations, where periodic (blind) retraining performed slightly better than retraining triggered by a drift detector and the best retraining window depended on the detector (Poenaru-Olaru et al., 2024); a recent preprint on streaming classifiers likewise reports periodic retraining outperforming error- and ADWIN-triggered policies except under recurring drift (Dasari, 2026). Those studies retrain on data whose labels or normality are known. The present study asks the same question for lightweight detectors on industrial-control data and adds the factor a deployed detector cannot avoid: whether the retraining data are selected with labels, by the detector's own predictions, or not at all.

Resource cost.

MLPerf Tiny measures accuracy, latency and energy of single inferences on constrained hardware, including an anomaly-detection task (Banbury et al., 2021). Comparable cost reporting for classical multivariate detectors on industrial data is rare. Here, cost is measured as single-reading latency, batch throughput, model size, load time and peak memory under explicit CPU and memory caps. Energy is not measured.

3. DATASETS

SKAB.

SKAB was recorded on a water-circulation testbed (pump, pipes, valves) with eight sensors sampled at 1 Hz: two accelerometers (RMS), motor current, pressure, water temperature, pump-housing temperature, voltage and volumetric flow rate (Katser & Kozitsin, 2020). It contains one 9,401-row anomaly-free run and 34 fault runs (46,860 rows in total) in three families: valve 1 (16 runs), valve 2 (4) and other faults (14). Faults are physically induced and labelled per second. Each file is an independent experiment.

SWaT.

SWaT is a six-stage water-treatment plant with 51 sensors and actuators sampled at 1 Hz (Goh et al., 2016; Mathur & Tippenhauer, 2016). The official release used here (iTrust, SUTD: SWaT.A1 & A2, December 2015) has a seven-day normal run (Normal_v1, 22–28 December, 495,000 rows, the first 30 minutes of start-up already removed) and an attack period (Attack_v0, 28 December–2 January, 449,919 rows) in which 54,621 rows (12.1%) belong to 36 attacks (37 rows spelled "A ttack" in the label column are counted as attacks). Following common practice (Li et al., 2019), the normal run is used from 22 December 22:00, six hours after the plant started. The 25 continuous sensors (flows, levels, pressures and chemical analysers) are used; the 26 discrete actuator states are not.

A widely downloaded public re-upload of the same data (Kaggle, vishala28/swat-dataset-secure-water-treatment-system, v3) was used in the initial analysis. Its normal.csv holds the normal run twice, with seven channels blank (including the analyser AIT201), plus the normal-labelled rows of the attack period, and its attack.csv holds only the attack rows. Rebuilding the standard split from it by de-duplicating timestamps reproduces the official row counts; Section 5.5 compares results from the two copies.

WADI.

WADI is a water-distribution testbed built as an extension of SWaT, with 127 points sampled at 1 Hz (Ahmed et al., 2017). The official A2 release (iTrust, November 2019) provides a 14-day normal run (784,571 rows, 25 September –7 October 2017) and a two-day attack file (172,803 rows, 9 –11 October) in which 9,977 rows (5.8%) belong to 14 attacks. Four columns are empty in both files and are

dropped; the first 21,600 normal rows are removed as for SWaT; the 67 points with more than ten distinct values in the training rows are used. The time column stores only minutes and seconds, so rows are indexed by their order; the row counter and the per-day row counts (86,400 on every complete day) confirm gap-free 1 Hz sampling in the attack file. The normal file has one gap of 264,000 rows (about 73 hours, 29 September to 2 October) removed in the release; it lies in the training portion, and rolling features and autoencoder windows are restarted at it so that no window joins readings from either side. Ignoring the gap changed no classical detector's AUROC by more than 0.001.

HAI.

HAI (version 21.03) comes from a hardware-in-the-loop testbed in which a steam-turbine process, a boiler process and a water-treatment process are coupled through a simulator, sampled at 1 Hz (Shin et al., 2020; Shin et al., 2021). It provides three normal training files (921,603 rows) and five test files (402,005 rows) in which 2.2% of rows belong to 50 scripted attacks. Of its 78 points, the 55 with more than ten distinct values in the training files are treated as continuous and used. HAI adds a second process type beyond water and a test set with a much lower anomaly rate than SWaT.

4. METHOD

4.1. Features

For each continuous sensor, the raw value, 5-second and 30-second causal rolling means, a 5-second rolling standard deviation and the first difference are computed (40 features on SKAB, 125 on SWaT, 335 on WADI, 275 on HAI). Features are computed on each complete recording *before* any rows are removed, so windows never join readings across a removed fault and, at test time, use only past readings. The initial analysis' elapsed-time feature was removed: its range differs between training (up to 9,960 s on SKAB) and test segments (at most 291 s), and it has no meaning for a monitor that runs continuously. Its effect is reported in Section 5.5. Wall-clock features remain excluded because they leak the identity of the recording session.

4.2. Detectors

Nine classical detectors are evaluated, all producing a continuous anomaly score: a maximum absolute z-score; PCA reconstruction error (95% of variance); Elliptic Envelope (robust MCD on ten whitened principal components, needed because the features are collinear); Isolation Forest (100 trees); One-Class SVM (RBF kernel, ν = 0.1); LOF (k = 20, novelty mode); and ECOD, COPOD and HBOS from PyOD (Zhao et al., 2019), with library defaults. On SWaT, WADI and HAI, One-Class SVM and LOF are fitted on a 20,000-row evenly spaced subsample of the training data because their cost grows quadratically; on WADI and HAI, ECOD and COPOD are fitted on a 100,000-row subsample because PyOD rescores test data jointly with the stored training data and the full set exceeded 16 GB of memory (test data are scored in 50,000-row chunks, so their WADI and HAI scores depend slightly on chunk size). All other detectors use every training row. The deep baseline is an LSTM autoencoder (one-layer encoder and decoder, 64 hidden units, ≈ 52k–58k parameters) on 30-second windows of raw scaled sensors, trained for 25 epochs with Adam; a row's score is the reconstruction error of the window ending at that row (Malhotra et al., 2016). All models are fitted on rows labelled normal, with one exception: under the SKAB reference protocol the first 400 rows of each run are used as given, and 2.1% of them are labelled anomalous. Table 1 lists the implementation details.

4.3. Threshold rules

A score becomes an alarm when it exceeds a threshold τ. Because the comparison should not depend on one rule, two rules are applied identically to every detector:

- **Label-free:** τ is the 99th percentile of scores on held-out *normal* data, so about 1% of normal readings are expected to raise an alarm. No anomaly labels are used.
- **Validation-MCC:** τ maximises MCC on a labelled validation set that contains anomalies, excluding thresholds that flag more than 99% of rows. This is semi-supervised calibration and is reported as such.

A third rule, maximising validation F1, is used only to show how much the initial analysis' choice of F1 mattered.

4.4. Splits

SKAB, pooled protocol.

As in the initial analysis, each fault run is split chronologically into its first 80% and last 20%. The training portions of 20% of the runs (whole runs, fixed seed) form a labelled validation set; the model is trained on the anomaly-free run plus the normal rows of the remaining training portions (24,930 rows); the last 20% of every run is the test set (7,503 rows, 30.0% anomalous). SKAB has no separate normal recording beyond the anomaly-free run, which is used for training, so the label-free threshold is set from the normal-labelled rows of the six validation runs' training portions (3,438 of their 5,562 rows; runs other/19 and valve1/1, 4, 6, 7, 13). Selecting these rows uses the per-

second labels; the threshold uses no anomalous rows, but on SKAB it is label-free only in that sense. On SWaT, WADI and HAI the held-out normal data come from normal-only recordings and need no labels.

SKAB, benchmark protocol.

The SKAB reference implementation trains a model per run on its first 400 rows and tests on the rest. This protocol is also reported, with the label-free threshold set from each run's own training scores, so results can be read against the SKAB leaderboard.

SWaT, standard protocol.

Detectors are trained on the normal run from 22 December 22:00 to 27 December 13:18 (400,680 rows); the rest of the run (74,520 rows) is held-out normal data for the label-free threshold; the first third of the attack period is the labelled validation set for the validation-MCC threshold. Threshold-free metrics and label-free results are reported on the full attack period; both threshold rules are also compared on its last two thirds, which neither rule has seen.

WADI.

The same design is used on WADI: training on the first 85% of the normal run, the last 15% as held-out normal data, the first third of the attack file for the validation-MCC threshold, and metrics on the whole file and on its last two thirds.

HAI.

The same design is used: training on the three normal files except the last 15% of the third, which is held-out normal data for the label-free threshold; the first test file is the labelled validation set for the validation-MCC threshold; metrics are reported on all five test files and, for the rule comparison, on test files 2–5. Bootstrap blocks are one-hour blocks within each test file.

4.5. Metrics and uncertainty

AUROC and average precision (AUPRC) summarise ranking quality independent of any threshold. At a threshold, precision, recall, F1, MCC, false-alarm rate on normal rows, event recall (the fraction of contiguous anomalous segments with at least one alarm) and median detection delay are reported. Detection delay is the time in seconds (rows at 1 Hz) from the first labelled row of an event to the first alarm within it, and its median is taken over detected events only; a detector that already alarms on most normal rows shows a delay near zero, so delay is informative only together with the false-alarm rate. MCC is

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

which is zero for any predictor independent of the labels, including one that flags every row; F1 for that predictor equals $2\pi/(1+\pi)$ for anomaly prevalence π (0.46 on the SKAB test set), which is why F1 alone can credit a detector that separates nothing (Chicco & Jurman, 2020). Precision is bounded by prevalence in the same way: with false-alarm rate f and recall r, $precision = \pi r/(\pi r + (1-\pi) f)$, so the same detector looks much worse in precision on SWaT (π = 0.121) than on a test set built with π = 0.315, as in the initial analysis. Test rows are strongly autocorrelated, so 95% confidence intervals are obtained by resampling whole blocks with replacement: the 34 test files on SKAB and non-overlapping one-hour blocks on SWaT, WADI and HAI. The same resamples are used for every detector, so paired differences are valid. Tables report a single fitted model (seed 0), and the intervals resample test blocks for that model, so they reflect test-set variability only, not training randomness. Training randomness is reported separately: Isolation Forest is repeated over ten seeds on every benchmark and the LSTM autoencoder over three seeds on SKAB. Overlapping intervals are not treated as a test; a claim that two detectors are indistinguishable rests on the paired bootstrap difference between them, computed on the same resamples, having a 95% interval that includes zero. Paired differences were computed for the two highest-AUROC detectors on each benchmark.

4.6. Drift and retraining

The retraining study simulates deployment on SWaT. An initial model is fitted on the first 50,000 normal rows after start-up (about 14 hours). The remainder of the normal week and then the attack period are streamed in one-hour batches; each batch is scored *before* it can be used for retraining. Four policies are compared: *static* (never retrain); *drift-triggered* (retrain when the mean two-sample Kolmogorov–Smirnov statistic between the batch and the current reference, $\acute{D} = \frac{1}{d}\sum_{j=1}^{d} \sup_x \left| F_j^{ref}(x) - F_j^{win}(x) \right|$ (Massey, 1951), exceeds the 95th percentile of its within-initial-window values); and *periodic*, with a retrain attempt every P batches for $P \in 1,2,3,4,6,12,24,48$, where P = 1 retrains after every batch. The periodic sweep traces detection skill against retraining budget, and the triggered policy is compared with that curve at its own number of successful refits. An *attempt* is a batch on which a policy fires; it becomes a *refit* only if the selected rows of the window exceed 1,000, otherwise the current model is kept. For every run the number of attempts, the number of refits,

and the CPU time spent on refits and on drift checks are recorded. Each retrain uses the last five batches, with rows selected in one of three ways: *oracle* (ground-truth normal rows, as in the initial analysis), *predicted-normal* (rows the current model did not flag) or *all* rows. After every fit the threshold is the 99th percentile of the fit-window scores. Isolation Forest and Elliptic Envelope are tested. The scaler is fitted once on the initial window. The KS statistic is computed over all 125 features between the current batch and a reference that is the initial 50,000-row window until the first refit and afterwards the rows the model was last refitted on. The trigger threshold ($\bar{D}$ = 0.317) is the 95th percentile of $\bar{D}$ between the initial window and each of its own 13 one-hour slices; because those slices are part of the reference, this calibration is conservative toward firing. The check runs on every batch, whether or not the previous attempt refitted. The five-batch window always includes the batch just scored. When an attempt finds 1,000 rows or fewer after selection, neither the model, its threshold nor the KS reference changes, and the next attempt again uses the five most recent batches. The KS statistic is used instead of its *p*-value because, with thousands of rows per batch, *p*-values vanish for negligible differences, whereas $\bar{D}$ is a bounded effect size.

4.7. Resource measurement

Each detector is fitted and timed in its own freshly started (spawned) process inside a Docker container run with `--cpus=1 --memory=1g` on an ARM64 (Apple M4) host, with BLAS threads limited to one; the container's control group confirmed a 1.0-CPU quota and a 1,024 MB memory limit. Reported are fit time, serialised model size, load time from the serialised form, single-reading latency over 1,000 calls after one warm-up call (median, 95th and 99th percentiles), batch throughput on 10,000 rows, and memory in two parts: the process's resident memory after the interpreter, libraries and data are loaded (*baseline*), and the peak increase above that baseline while fitting and scoring (*model memory*, from the kernel's high-water mark reset after loading). The container-wide peak and memory events are read from the control group. The cost of recomputing the rolling features for one new reading from a 30-row buffer is reported separately. This is a capped container, not an edge device: a physical device's slower cores, thermal limits and I/O are not represented, so latencies are lower bounds.

5. RESULTS

5.1. SKAB under the pooled protocol

Table 2 and Fig. 1 (left) report SKAB under the pooled protocol. Every detector is close to chance. The highest AUROC is Isolation Forest's 0.575 (95% CI 0.504–0.645), and COPOD's interval (0.507–0.621) also excludes 0.5; both do so narrowly, which is weak evidence of above-chance ranking. Over ten seeds Isolation Forest's AUROC is 0.575 ± 0.018 (range 0.550–0.603), so its margin over chance depends on the seed as well as on the test files. The paired comparison of the two highest-scoring detectors (Isolation Forest and COPOD) did not establish a significant difference (0.012, CI −0.066 to 0.092); other pairs were not tested. The LSTM autoencoder is below chance (AUROC 0.442, three seeds within ± 0.001). With a calibrated threshold the best MCC is 0.113 (COPOD; CI 0.045–0.190). The label-free rule gives MCC within ± 0.05 of zero for every detector: thresholds learned from normal data in this pooled setting do not separate faults.

5.2. SWaT under the standard protocol

Table 3 and Fig. 1 report SWaT (official release, 25 sensors). All ten detectors rank attack rows well (AUROC 0.811–0.856) and their CIs overlap almost completely. The paired comparison of the two highest-scoring detectors (ECOD and COPOD) gives a difference of 0.007 (CI −0.003 to 0.017), so no significant difference is established between them; equivalence among all ten detectors was not formally tested. Isolation Forest's AUROC is 0.824 for the seed-0 model in the table and 0.819 ± 0.005 over ten seeds. The LSTM autoencoder ranks attacks as well as the classical detectors (AUROC 0.822, AUPRC 0.731).

The detectors separate once a threshold is applied. With the label-free rule, which targets 1% false alarms on held-out normal-week data, COPOD reaches MCC 0.628 (CI 0.488–0.730), Isolation Forest 0.539 and HBOS 0.463, with false-alarm rates of 0.1–11.8% on the attack period's normal rows. The *z*-score, One-Class SVM, LOF, PCA reconstruction and the LSTM autoencoder instead raise alarms on 42–75% of the attack period's *normal* rows, although their thresholds gave exactly 1% on held-out normal-week data. Their scores shift between the normal week and the attack week independently of the attacks. The shift comes from SWaT's chemical analysers: relative to the training range, AIT402 is out of range on 98% of the attack period's normal rows, AIT201 on 73% and AIT501–504 on 53–58%. Detectors that combine all features through distances or reconstructions inherit any single feature's shift; detectors that score each feature's rarity separately (COPOD, ECOD, HBOS) or split on one feature at a time (Isolation Forest) bound its influence. Elliptic Envelope shows the opposite failure, with very few alarms (MCC 0.005). Among the detectors whose label-free false-alarm rate stayed low, the median delay to the first alarm in a detected attack was 12.5 s for Isolation Forest, 20.5 s for

HBOS, 66 s for Elliptic Envelope and 41–111 s for ECOD and COPOD, which also detected the fewest attacks (event recall 0.17–0.26); the zero delays of the high-false-alarm detectors reflect alarms that were already firing. When a labelled third of the attack period is used to calibrate thresholds, MCC on the remaining two thirds rises sharply for the detectors whose label-free thresholds failed (One-Class SVM 0.822, *z*-score 0.820, LOF 0.742), but not for PCA reconstruction (0.078) or the LSTM autoencoder (0.101).

5.3. HAI

Table 4 reports HAI. Unlike SWaT, the detectors do separate. PCA reconstruction (AUROC 0.929, CI 0.899–0.954), LOF (0.920) and the LSTM autoencoder (0.916) lead, followed by One-Class SVM and the *z*-score (0.895); the difference between the top two is 0.008 (CI −0.009 to 0.025). HBOS (0.728) and Elliptic Envelope (0.663, CI 0.608–0.716) are clearly worse, and Isolation Forest sits in between (0.837 for the seed-0 model in the table, 0.815 ± 0.009 over ten seeds; the seed-to-seed spread is of the same order as its gap to COPOD and ECOD). Label-free thresholds transfer well on HAI: held-out normal data from the training period predicted the test-period false-alarm rate closely (0.4–1.6% for every classical detector against a 1% target; 9.1% for the LSTM autoencoder), and the leading five detectors raised alarms in all 50 attacks, with a median delay of 1.5–5 s (Isolation Forest 27 s, Elliptic Envelope 38 s). With these thresholds MCC is 0.597–0.627 for PCA reconstruction, LOF, One-Class SVM and the *z*-score. Calibrating on a labelled test file raises MCC on the remaining files only slightly (by 0.05–0.06) for the leaders, but by 0.33 for the LSTM autoencoder, whose label-free threshold was too low.

Across the two plants, the detectors that lead differ: PCA reconstruction and LOF, which failed under label-free thresholds on SWaT, lead on HAI, where no analyser drift of the SWaT kind occurs and thresholds transfer; Elliptic Envelope trails on both.

5.4. WADI

WADI is the hardest benchmark (Table 5). With all 67 points, the per-feature detectors lead (HBOS AUROC 0.742, Isolation Forest 0.720, COPOD and ECOD 0.71; Isolation Forest 0.725 ± 0.016 over ten seeds; the difference between HBOS and Isolation Forest is 0.021, CI −0.019 to 0.064), while the *z*-score, One-Class SVM, LOF, PCA reconstruction, Elliptic Envelope and the LSTM autoencoder are at chance (AUROC 0.46–0.52). These six detectors raise alarms on 55.8–56.1% of the attack file's normal rows under the label-free rule, and the near-identical rate points to a single cause: the analyser 2B_AIT_002_PV is outside its training range on 55.5% of those rows (2A_AIT_004_PV on 26%, 1_AIT_003_PV on 7%). Repeating the experiment without these three analysers removes the effect: every detector's label-free false-alarm rate falls to 0.6–1.4%, AUROC moves into a common band of 0.649–0.748 with overlapping CIs, and label-free MCC reaches 0.329 for the LSTM autoencoder and 0.18–0.23 for LOF, the *z*-score, PCA reconstruction and One-Class SVM. This ablation is a post-hoc diagnostic: the three analysers were chosen after inspecting their ranges in the attack file, so it shows that the sensor set can decide whether thresholds transfer, but it is not a procedure that a deployment could apply in advance. A label-free version would compare recent sensor ranges with the training ranges and exclude or down-weight drifting channels; that was not evaluated. Absolute performance on WADI remains low, as other studies also find.

5.5. How much protocol choices change the conclusions

Split definition (SWaT).

Scoring the same detectors with the same code on the re-upload's layout (the split of the initial analysis: train on the normal rows between attacks, test on 31.5% attacks) gives AUROC 0.840–0.934 and reorders the detectors: HBOS ranks first there and last under the standard split with the same 24 sensors (Fig. 2, left). Under that layout the training data come from the same five days as the test attacks, so the normal-week to attack-week shift described above never appears in evaluation.

Data copy and sensor set (SWaT).

With the same 24 sensors, the standard split rebuilt from the public re-upload and the official release give practically identical results (every detector's AUROC within 0.001, label-free MCC within 0.008), so the reconstruction is faithful and the re-upload's problem lies in how its files are organised. The one real difference is the 25th sensor, the analyser AIT201, which the re-upload leaves blank. Adding it leaves AUROC in the same narrow range (0.81–0.86) but changes which thresholds survive: the *z*-score's label-free false-alarm rate rises from 1.4% to 42% (MCC 0.716 to 0.215) and One-Class SVM's from 3.8% to 58% (MCC 0.660 to 0.184), while COPOD (MCC 0.592 to 0.628) and Isolation Forest (0.630 to 0.539) are barely affected. A single sensor therefore decides whether the simplest detector looks dependable.

Threshold rule (SWaT).

The initial analysis tuned one contamination value on Isolation Forest and applied it to every model. On the same

layout, per-model validation-MCC thresholds (with the revised detector settings of Section 4) raise the *z*-score's MCC from 0.180 to 0.844, PCA reconstruction's from 0.002 to 0.834 and LOF's from 0.105 to 0.830, while Elliptic Envelope, the initial winner at 0.766, moves only to 0.810. The initial ranking therefore reflected mainly the shared threshold, not the detectors.

Training protocol (SKAB).

Under the SKAB reference protocol (one model per run, trained on its first 400 rows), the same detectors are far better than under pooled training: mean per-run AUROC reaches 0.794 (PCA reconstruction), 0.785 (LOF) and 0.771 (One-Class SVM), against 0.489–0.575 pooled (Fig. 2, right; Table 6). The ranking inverts: Isolation Forest, the best pooled detector, is among the weakest per run, and ECOD and COPOD fall to chance because, with only 400 training rows and a 99th-percentile threshold, they almost never alarm. Of the first 400 rows, 2.1% are labelled anomalous, so this protocol trains on slightly contaminated data, as in the reference implementation. Absolute numbers are not directly comparable with the SKAB leaderboard (best F1 0.78 at a 13.6% false-alarm rate for a convolutional autoencoder), because features, thresholds and post-processing differ; detectors here reach similar F1 (0.72–0.73) only at much higher false-alarm rates (47–71%), while Elliptic Envelope offers a more balanced operating point (F1 0.644, false alarms 20.2%, MCC 0.361).

Threshold rule (SKAB).

On the pooled SKAB test set, Isolation Forest's MCC is 0.005 with the label-free rule, 0.097 with validation-MCC and 0.118 with validation-F1; the F1 rule buys its MCC with recall 0.71 at higher false-alarm rates. Differences of this size are within the CIs.

Feature choices.

Removing the elapsed-time feature and computing features before dropping fault rows changed SKAB AUROC by at most 0.02 (Isolation Forest 0.591 initial pipeline, 0.555 with elapsed time, 0.575 revised), well within the CIs; these corrections matter for validity, not for the numbers.

6. DRIFT AND RETRAINING

The retraining study uses the official SWaT release with all 25 sensors.

Does normal operation drift?

Streaming SWaT's normal week through a static Isolation Forest fitted on its first 14 hours, the share of normal readings that raise an alarm shows two transient spikes in the first 30 hours (up to 45%), stays near the design rate of 1% until about hour 48, and then jumps to 33–100% for the rest of the week (mean 10% over the first 24 streamed hours, 86% over the last 24; Fig. 3, left). The change is a step, consistent with a change in operating regime, rather than a slow trend. The Elliptic Envelope fitted on the same window does not show this: its false-alarm rate over the normal week is 0.9%. Drift is therefore real, but whether it matters depends on the detector. Isolation Forest operates on all 125 features, whereas the Elliptic Envelope operates on ten whitened principal components; a plausible, untested explanation is that the level changes of individual sensors fall mostly outside those components.

Retraining budget.

A retrain *attempt* is a batch on which the policy fires; it becomes a *refit* only if the selected five-hour window holds more than 1,000 rows, otherwise the current model is kept. To compare policies at equal cost, the periodic policy was run every $P \in 1,2,3,4,6,12,24,48$ batches, which traces how detection skill grows with the number of refits; the KS-triggered policy is then read against that curve (Table 7, Fig. 3).

With labelled retraining data.

When retraining rows are selected with ground-truth labels (as in the initial analysis), Isolation Forest's skill over the attack period rises steadily with the number of refits: MCC 0.165 static (75% of the attack period's normal rows flagged), 0.288 with 5 refits, 0.404 with 40, 0.478 with 80, 0.502 with 120 and 0.567 when refitting after every batch (239). The KS-triggered policy reached 0.549 with 89 refits (94 attempts), above the periodic curve at that budget (0.478 with 80 refits, 0.502 with 120) and close to refitting after every batch, so in this stream triggering saved refits. Because the stream is a single realisation, this comparison has no confidence interval and does not establish that the trigger is better in general. For the Elliptic Envelope, no budget changes the outcome: MCC 0.705–0.732 at every periodic budget and 0.709 for the KS policy, against 0.724 for the static model.

Without labels, retraining can fail.

A deployed detector does not know which recent rows are normal. When Isolation Forest retrains on the rows it did *not* flag, it enters a self-reinforcing failure: once it flags most rows, too few remain to refit on, attempts stop turning into refits (224 attempts gave 24 refits under the KS policy), and the detector ends up flagging everything (MCC 0.000–0.069 at every budget). Retraining on all recent rows avoids that collapse but absorbs the attacks into the model of normal behaviour: recall falls to 0.27–0.45 and MCC to 0.084–

0.255 whenever refits are frequent (40 or more). The Elliptic Envelope tolerates predicted-normal selection at every budget (MCC 0.703–0.749). With all-rows retraining it loses skill only when refits are frequent (MCC 0.329–0.530 with 40 or more refits, 0.678–0.718 with 20 or fewer), because each frequent refit takes in a larger share of the ongoing attacks. The benefit of adaptation reported in the initial analysis therefore depended on labels that a deployment would not have.

Cost.

On the host, an Isolation Forest refit on five hours of data took about 0.07 s and an Elliptic Envelope refit about 0.9 s. The KS check (125 features against a reference of up to 50,000 rows) cost 53–98 s of CPU over the 244 batches in the label-selected and all-rows runs (0.2–0.4 s per batch) and 386 s in the collapsing predicted-normal run. For Isolation Forest the checks alone cost more than five times as much as refitting after every batch (98 s against 18 s, oracle selection), and every-batch refitting gave the best detection, so the refits a trigger saves do not pay for the check. A trigger can pay only when refits are expensive relative to the check, as for the Elliptic Envelope, which in this stream did not need retraining at all.

A note on the Elliptic Envelope's instability.

In this streaming setting the static Elliptic Envelope, fitted on 14 hours, reaches MCC 0.724, yet in Table 3, fitted on the full normal week with its threshold set on the week's last 15%, its label-free MCC is 0.005. The two runs differ only in the training window and threshold-calibration data. One consistent reading is that drift within the normal week inflates the end-of-week scores used for calibration, producing a threshold too high to fire. Either way, a detector whose performance swings this much with the calibration window is fragile in deployment, a property a single benchmark number hides.

7. RESOURCE COST PER READING

Table 8 and Fig. 4 report the cost of each detector fitted on SWaT-shaped data inside the one-CPU, 1 GB ARM64 container. Single-reading latency spans more than five orders of magnitude. The *z*-score scores a reading in 0.002 ms; PCA reconstruction, HBOS, Elliptic Envelope and One-Class SVM take 0.05–0.16 ms; LOF 0.89 ms; Isolation Forest 2.3 ms, dominated by per-call overhead across its 100 trees, since its batch throughput is about 850,000 rows/s. ECOD and COPOD, as implemented in PyOD, keep their training data and rescore against it on every call: 698 and 739 ms per reading and 235 MB models. Above a baseline of about 444 MB (Python, libraries and data), peak resident memory rose by 71–228 MB while fitting and scoring, except for ECOD and COPOD (516 and 527 MB); this increase includes transient scoring buffers as well as the stored model. The container's memory use reached its 1 GB limit only during the ECOD and COPOD runs: the kernel recorded 3,127 reclaim events at the limit and no out-of-memory kills, so the cap was enforced and both completed. On SKAB-shaped data (40 features) the pattern is the same at smaller scale (Isolation Forest 2.3 ms; ECOD and COPOD 118–125 ms, 39 MB models, about 260 MB peak increase). The LSTM autoencoder takes 0.78 ms per 30-second window with 58k parameters, but needs a deep-learning runtime (peak process memory 335 MB for inference alone). Recomputing the rolling features for one new reading from a 30-row buffer with the pipeline's pandas code costs 3.2 ms, more than any detector except ECOD and COPOD, so in a streaming deployment incremental feature updates would matter more than the choice among the fast detectors. These are lower bounds for a physical device. (An earlier measurement in forked child processes over-counted memory, because a forked child's resident set includes pages shared with its parent; it is not used here.)

8. DISCUSSION

Table 9 condenses what each protocol choice did to the measured outcome and what it implies for someone selecting or deploying a detector.

Ranking quality is not the bottleneck; thresholds and sensors are.

On SWaT, ten detectors that span statistical, distance, density, tree, copula and recurrent families lie within 0.045 AUROC of each other with overlapping intervals (the paired difference between the top two includes zero; equivalence of all ten was not formally tested), and on WADI they fall into a common band once three drifting analysers are removed. This agrees with large benchmark studies that find no consistent winner and strong simple baselines (Audibert et al., 2022; Sarfraz et al., 2024). What separates detectors in use is whether a threshold set on normal data still works later. On both water plants the decisive factor was the chemical analysers, whose readings left their training range between the normal and the attack recordings: detectors that combine all features through distances or reconstructions inherited that shift and raised alarms on roughly half or more of normal readings, while detectors that bound each feature's influence (COPOD, ECOD, HBOS, Isolation Forest) did not. On HAI, where no comparable shift occurred, thresholds transferred and the distance-based detectors led. Which detector is best is therefore a property of the plant, its sensors and the protocol together. For a

deployment this suggests two practical checks before choosing a detector: compare recent sensor ranges with the training ranges, and evaluate thresholds on normal data recorded after the training period.

Why the initial conclusions changed.

Three protocol choices in the initial analysis each favoured a particular answer. Training on the attack period's own normal rows removed the normal-week to attack-week shift from the evaluation. A contamination value tuned on Isolation Forest and shared by all detectors made detectors with differently scaled scores look poor. Retraining on ground-truth-normal rows made adaptation look free of risk. None of these choices is unusual, and each was documented in the code. Their effect became visible only when the same detectors were rerun under alternative protocols. Studies that compare detectors should therefore report at least one alternative threshold rule and state where their training and test data come from in time.

What adaptation needs.

Normal operation does drift, and some detectors are much more sensitive to it than others. Where retraining helps, it helps only if the retraining data are clean; with label-free selection Isolation Forest either collapsed into flagging everything or learned the attacks as normal. In practice this argues for one of three things: retraining data vetted by an operator; a slow-changing detector (here, the Elliptic Envelope on whitened components), whose static form was as good as any adaptive variant; or adaptation of the threshold rather than the model. For a PHM system this means treating retraining as a maintenance action with its own check, not as an automatic background step. For cheap models such as Isolation Forest, a drift test can cost more than the retrain it is meant to save.

Cost.

Single-reading latencies span almost six orders of magnitude, from microseconds (*z*-score) to a median of 698 –739 ms (99th percentile up to 1.1 s) for ECOD and COPOD (in PyOD on SWaT, which store and rescore against their training data), and model sizes from 1 KB to 235 MB. The batch-amortised figures used in the initial analysis (around 0.002 ms per row for Isolation Forest) understate the real single-reading cost (about 2.3–2.8 ms) by three orders of magnitude, because per-call overhead dominates. For a monitor that scores one reading per second, all detectors except possibly ECOD and COPOD fit comfortably; the deciding constraints are memory and threshold stability, not speed.

9. THREATS TO VALIDITY AND LIMITATIONS

- **Dataset versions.** SWaT and WADI results use the official iTrust releases (SWaT A1 & A2, December 2015; WADI A2, 2019). Later SWaT and WADI campaigns, with different attacks and plant configurations, were not evaluated. The drift study and the cost benchmark used SWaT; the cost benchmark used the re-upload's 24 sensors (120 features rather than 125), which is not expected to change timing materially.
- **Process coverage.** SKAB and SWaT are water processes; HAI adds turbine and boiler processes but through a hardware-in-the-loop simulator. Rotating machinery under real wear, electrical faults and naturally occurring (rather than induced or scripted) anomalies are not covered; run-to-failure machinery data are a natural next step. The drift and cost studies were run on SWaT only.
- **Detector configurations.** Hyperparameters other than thresholds were fixed at common defaults; One-Class SVM and LOF were trained on a 20,000-row subsample on SWaT, WADI and HAI, and ECOD and COPOD on a 100,000-row subsample on WADI and HAI; the LSTM autoencoder is small and was run with one seed on SWaT. Larger or tuned deep models were not tested.
- **Metrics.** Event recall and median delay are reported, but range-aware metrics such as VUS-PR (Paparrizos et al., 2022) were not computed. Bootstrap blocks (files, hours) reduce but do not remove dependence between blocks.
- **Drift design.** One stream, one window length (five hours), one KS threshold rule and two detectors were studied; ADWIN-style detectors and threshold-only adaptation were not compared. Policies are compared without confidence intervals because the stream is a single realisation; the trigger-versus-schedule comparison holds for this stream and these settings only.
- **Resource measurements.** A Docker container capped at one CPU and 1 GB on an Apple M4 host is not a physical edge device: slower cores, thermal limits and real I/O are absent, so latencies are lower bounds. Memory is reported as the peak RSS increase above a measured baseline and includes transient scoring buffers. Energy was not measured.
- **SKAB protocols.** The SKAB reference protocol trains on rows of which 2.1% are labelled anomalous, and its label-free threshold is set from

each run's own training scores; results are not directly comparable with the SKAB leaderboard, whose features, thresholds and post-processing differ.

- **Search scope.** The literature search used a general web search and publisher pages rather than database-level systematic queries; statements that a comparison was not found are limited to that search.

10. CONCLUSION

Evaluating ten lightweight and deep detectors on SKAB, SWaT, WADI and HAI under documented, standard protocols answered the four research questions as follows. (RQ1) On SWaT all ten detectors reach AUROC 0.81–0.86 with overlapping intervals (the top two not significantly different; equivalence of all ten not formally tested), and on WADI they fall into a common band once three drifting analysers are removed; on HAI they separate, with PCA reconstruction and LOF leading (0.93, 0.92) and Elliptic Envelope trailing (0.66); on SKAB with pooled training, all are close to chance and the paired comparison of the two highest-scoring detectors did not establish a significant difference. (RQ2) Protocol choices change the conclusions more than detector choice does: the SWaT split definition reorders the detectors, a shared contamination value had made the *z*-score look useless (MCC 0.18 against 0.84 with its own threshold), SKAB's per-run protocol inverts the pooled ranking, and a single analyser decides whether label-free thresholds survive (the *z*-score's false alarms 1.4% without AIT201, 42% with it). With label-free thresholds on the official SWaT data, COPOD, Isolation Forest and HBOS were the most dependable (MCC 0.46–0.63). (RQ3) In the single stream tested, normal operation drifts enough to break a static Isolation Forest; retraining restores it only when the retraining data are known to be normal, and label-free selection makes it collapse or absorb the attacks, whereas Elliptic Envelope stays stable under predicted-normal selection and degrades only with frequent all-rows retraining; drift-triggered retraining saved some refits but cost more in drift checks than it saved. (RQ4) Median single-reading costs range from microseconds to about 740 ms (99th percentile up to 1.1 s) and model sizes from kilobytes to hundreds of megabytes under a one-CPU, 1 GB cap; the per-reading feature update (3.2 ms) costs more than most detectors.

The initial analysis reported dataset-specific winners, a large benefit from drift adaptation, and sub-millisecond inference; none of these survived re-examination. What did survive is a clearer and more useful message for practitioners: choose the detector for its threshold stability and retraining needs rather than its benchmark rank, and report the protocol in enough detail that a reader can tell which of these the numbers measure. Next steps are later SWaT and WADI campaigns, data from rotating machinery under real wear, drift-aware handling of analyser channels, threshold-only adaptation, and measurement on physical edge hardware.

DATA AND CODE AVAILABILITY

Code, configuration and all result files are available at https://github.com/sarjanshah14/SensorGuard-Research (experiments in experiments/revision/, results in results/revised/). SKAB and HAI are public (Katser & Kozitsin, 2020; Shin et al., 2021). SWaT and WADI must be requested from iTrust, SUTD, and are not redistributed; the comparison with the public SWaT re-upload is documented in the code. Software used: scikit-learn (Pedregosa et al., 2011), PyOD (Zhao et al., 2019) and PyTorch.

DISCLOSURE OF INTERESTS

The authors have no competing interests to declare.

REFERENCES

Ahmad, S., Lavin, A., Purdy, S., & Agha, Z. (2017). Unsupervised real-time anomaly detection for streaming data. *Neurocomputing*, 262, 134–147. doi:10.1016/j.neucom.2017.04.070

Ahmed, C. M., Palleti, V. R., & Mathur, A. P. (2017). WADI: A water distribution testbed for research in the design of secure cyber physical systems. In *Proc. 3rd Int. Workshop on Cyber-Physical Systems for Smart Water Networks (CySWater)* (pp. 25–28). doi:10.1145/3055366.3055375

Audibert, J., Michiardi, P., Guyard, F., Marti, S., & Zuluaga, M. A. (2020). USAD: Unsupervised anomaly detection on multivariate time series. In *Proc. 26th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining*. doi:10.1145/3394486.3403392

Audibert, J., Michiardi, P., Guyard, F., Marti, S., & Zuluaga, M. A. (2022). Do deep neural networks contribute to multivariate time series anomaly detection? *Pattern Recognition*, 132, 108945. doi:10.1016/j.patcog.2022.108945

Banbury, C., et al. (2021). MLPerf Tiny benchmark. In *Proc. NeurIPS Datasets and Benchmarks Track*. arXiv:2106.07597

Bifet, A., & Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In *Proc. SIAM Int. Conf. on Data Mining* (pp. 443–448). doi:10.1137/1.9781611972771.42

Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. In *Proc. ACM SIGMOD Int. Conf. on Management of Data* (pp. 93–104). doi:10.1145/342009.335388

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58. doi:10.1145/1541880.1541882

Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21, 6. doi:10.1186/s12864-019-6413-7

Dasari, S. (2026). When to retrain: An empirical study of retraining policies for streaming ML under concept drift, budget, and latency constraints. arXiv preprint arXiv:2608.19488.

Deng, A., & Hooi, B. (2021). Graph neural network-based anomaly detection in multivariate time series. In *Proc. AAAI Conf. on Artificial Intelligence*, 35(5), 4027–4035. doi:10.1609/aaai.v35i5.16523

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 1–37. doi:10.1145/2523813

Garg, A., Zhang, W., Samaran, J., Savitha, R., & Foo, C.-S. (2021). An evaluation of anomaly detection and diagnosis in multivariate time series. *IEEE Transactions on Neural Networks and Learning Systems*. doi:10.1109/TNNLS.2021.3105827

Goh, J., Adepu, S., Junejo, K. N., & Mathur, A. (2016). A dataset to support research in the design of secure water treatment systems. In *Proc. 11th Int. Conf. on Critical Information Infrastructures Security (CRITIS)* (pp. 88–99). doi:10.1007/978-3-319-71368-7_8

Goldstein, M., & Uchida, S. (2016). A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data. *PLoS ONE*, 11(4), e0152173. doi:10.1371/journal.pone.0152173

Han, S., Hu, X., Huang, H., Jiang, M., & Zhao, Y. (2022). ADBench: Anomaly detection benchmark. In *Advances in Neural Information Processing Systems 35 (Datasets and Benchmarks Track)*. arXiv:2206.09426

Hundman, K., Constantinou, V., Laporte, C., Colwell, I., & Soderstrom, T. (2018). Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. In *Proc. 24th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining*. doi:10.1145/3219819.3219845

Katser, I. D., & Kozitsin, V. O. (2020). *Skoltech Anomaly Benchmark (SKAB)* [Data set]. Kaggle. doi:10.34740/KAGGLE/DSV/1693952

Kim, S., Choi, K., Choi, H.-S., Lee, B., & Yoon, S. (2022). Towards a rigorous evaluation of time-series anomaly detection. In *Proc. AAAI Conf. on Artificial Intelligence*, 36(7), 7194–7201. doi:10.1609/aaai.v36i7.20680

Lee, W.-J. (2017). Anomaly detection and severity prediction of air leakage in train braking pipes. *International Journal of Prognostics and Health Management*, 8(3). doi:10.36001/ijphm.2017.v8i3.2662

Li, D., Chen, D., Jin, B., Shi, L., Goh, J., & Ng, S.-K. (2019). MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks. In *Proc. Int. Conf. on Artificial Neural Networks (ICANN)*. arXiv:1809.04758

Li, Z., Zhao, Y., Hu, X., Botta, N., Ionescu, C., & Chen, G. H. (2023). ECOD: Unsupervised outlier detection using empirical cumulative distribution functions. *IEEE Transactions on Knowledge and Data Engineering*, 35(12), 12181–12193. doi:10.1109/TKDE.2022.3159580

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proc. 8th IEEE Int. Conf. on Data Mining* (pp. 413–422). doi:10.1109/ICDM.2008.17

Liu, Q., & Paparrizos, J. (2024). The elephant in the room: Towards a reliable time-series anomaly detection benchmark. In *Advances in Neural Information Processing Systems 37 (Datasets and Benchmarks Track)* (pp. 108231–108261).

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363. doi:10.1109/TKDE.2018.2876857

Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. (2016). LSTM-based encoder-decoder for multi-sensor anomaly detection. *ICML Anomaly Detection Workshop*. arXiv:1607.00148

Massey, F. J. (1951). The Kolmogorov-Smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253), 68–78. doi:10.1080/01621459.1951.10500769

Mathur, A. P., & Tippenhauer, N. O. (2016). SWaT: A water treatment testbed for research and training on ICS security. In *Proc. Int. Workshop on Cyber-physical Systems for Smart Water Networks (CySWater)* (pp. 31–36). doi:10.1109/CySWater.2016.7469060

Mukherjee, R. (2021). A modified energy statistic for unsupervised anomaly detection. *International Journal of Prognostics and Health Management*, 12(1). doi:10.36001/ijphm.2021.v12i1.1323

Paparrizos, J., Boniol, P., Palpanas, T., Tsay, R. S., Elmore, A., & Franklin, M. J. (2022). Volume under the surface: A new accuracy evaluation measure for time-series anomaly detection. *Proceedings of the VLDB*

Endowment, 15(11), 2774–2787. doi:10.14778/3551793.3551830

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Poenaru-Olaru, L., Karpova, N., Cruz, L., Rellermeyer, J. S., & van Deursen, A. (2024). Is your anomaly detector ready for change? Adapting AIOps solutions to the real world. In *Proc. IEEE/ACM 3rd Int. Conf. on AI Engineering – Software Engineering for AI (CAIN)* (pp. 222–233). doi:10.1145/3644815.3644961

Rabanser, S., Günnemann, S., & Lipton, Z. (2019). Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems 32*. arXiv:1810.11953

Rousseeuw, P. J., & Van Driessen, K. (1999). A fast algorithm for the minimum covariance determinant estimator. *Technometrics*, 41(3), 212–223. doi:10.1080/00401706.1999.10485670

Sarfraz, M. S., Chen, M.-Y., Layer, L., Peng, K., & Koulakis, M. (2024). Position: Quo vadis, unsupervised time series anomaly detection? In *Proc. 41st Int. Conf. on Machine Learning*, PMLR 235, 43461–43476.

Schmidl, S., Wenig, P., & Papenbrock, T. (2022). Anomaly detection in time series: A comprehensive evaluation. *Proceedings of the VLDB Endowment*, 15(9), 1779–1797. doi:10.14778/3538598.3538602

Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., & Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7), 1443–1471. doi:10.1162/089976601750264965

Shin, H.-K., Lee, W., Yun, J.-H., & Kim, H. (2020). HAI 1.0: HIL-based augmented ICS security dataset. In *Proc. 13th USENIX Workshop on Cyber Security Experimentation and Test (CSET)*.

Shin, H.-K., Lee, W., Yun, J.-H., & Min, B.-G. (2021). Two ICS security datasets and anomaly detection contest on the HIL-based augmented ICS testbed. In *Proc. Cyber Security Experimentation and Test Workshop (CSET '21)*.

Shyu, M.-L., Chen, S.-C., Sarinnapakorn, K., & Chang, L. (2003). A novel anomaly detection scheme based on principal component classifier. In *Proc. IEEE ICDM Workshop on Foundations and New Directions of Data Mining* (pp. 172–179).

Tatbul, N., Lee, T. J., Zdonik, S., Alam, M., & Gottschlich, J. (2018). Precision and recall for time series. In *Advances in Neural Information Processing Systems 31*. arXiv:1803.03639

Tuli, S., Casale, G., & Jennings, N. R. (2022). TranAD: Deep transformer networks for anomaly detection in multivariate time series data. *Proceedings of the VLDB Endowment*, 15(6), 1201–1214. doi:10.14778/3514061.3514067

Weiss, B., & Brundage, M. (2021). Measurement and evaluation for prognostics and health management (PHM) for manufacturing operations: Summary of an interactive workshop highlighting PHM trends. *International Journal of Prognostics and Health Management*, 12(1). doi:10.36001/ijphm.2021.v12i1.2653

Wu, R., & Keogh, E. (2023). Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, 35(3), 2421–2429. doi:10.1109/TKDE.2021.3112126

Zhao, Y., Nasrullah, Z., & Li, Z. (2019). PyOD: A Python toolbox for scalable outlier detection. *Journal of Machine Learning Research*, 20(96), 1–7.

Detector	Configuration	Score (higher = more anomalous)
z-score	training mean and s.d. per feature	$\max_j \lvert x_j - \mu_j \rvert / \sigma_j$
PCA-recon	components for 95% of variance	squared reconstruction error
Elliptic Envelope	10 whitened PCs; robust MCD (default support)	Mahalanobis distance
Isolation Forest	100 trees, default subsample, seeds 0–9	−score_samples
One-Class SVM	RBF, $\nu = 0.1$, gamma = scale	−decision_function
LOF	$k = 20$, novelty mode	−score_samples
ECOD, COPOD, HBOS	PyOD defaults	decision_function
LSTM-AE	1-layer LSTM enc./dec., 64 units, window 30, Adam, lr 10^{-3},	window MSE
	batch 256, 25 epochs, stride 2 (4 on HAI); raw sensors, own scaler	
Software: Python 3.14 (host) / 3.12 (container), NumPy 2.4.6, SciPy 1.17.1, pandas 3.0.3,		
scikit-learn 1.9.0, PyOD 3.6.7, PyTorch 2.12.1. Seeds: 0 unless stated; bootstrap: NumPy default_rng(0).		

Table 1. Implementation details. All classical detectors receive the same standardised feature matrix (StandardScaler fitted on training rows only); scores are oriented so that higher means more anomalous.

Detector	AUROC [95% CI]	AUPRC	MCC LF	MCC VM [95% CI]	Event rec. VM	Delay VM (s)
Isolation Forest	0.575 [0.504, 0.645]	0.338	0.005	0.097 [−0.026, 0.233]	0.94	3
COPOD	0.558 [0.507, 0.621]	0.320	−0.042	0.113 [0.045, 0.190]	1.00	0
ECOD	0.535 [0.476, 0.601]	0.315	−0.016	0.066 [−0.010, 0.159]	1.00	0
Elliptic Envelope	0.520 [0.459, 0.581]	0.305	−0.050	0.051 [−0.079, 0.182]	0.45	0
z-score	0.513 [0.477, 0.554]	0.307	−0.051	−0.032 [−0.102, 0.039]	0.48	8.5
HBOS	0.510 [0.420, 0.602]	0.315	0.041	0.012 [−0.116, 0.152]	0.85	0.5
One-Class SVM	0.498 [0.463, 0.533]	0.300	−0.035	−0.042 [−0.106, 0.009]	0.73	10
PCA-recon	0.489 [0.455, 0.517]	0.294	−0.046	−0.047 [−0.104, −0.005]	0.76	17
LOF	0.489 [0.450, 0.533]	0.299	0.007	−0.021 [−0.073, 0.032]	0.82	14
LSTM-AE	0.442 [0.392, 0.500]	0.267	−0.075	−0.078 [−0.165, −0.007]	0.27	33

Table 2. SKAB, pooled protocol (test: last 20% of each of 34 runs, 7,503 rows, 30.0% anomalous). Seed-0 models. AUROC and validation-MCC (VM) MCC with 95% block-bootstrap CIs over files (test-block variability only); label-free (LF) MCC is a point estimate. Event recall and median detection delay use VM; 33 fault events.

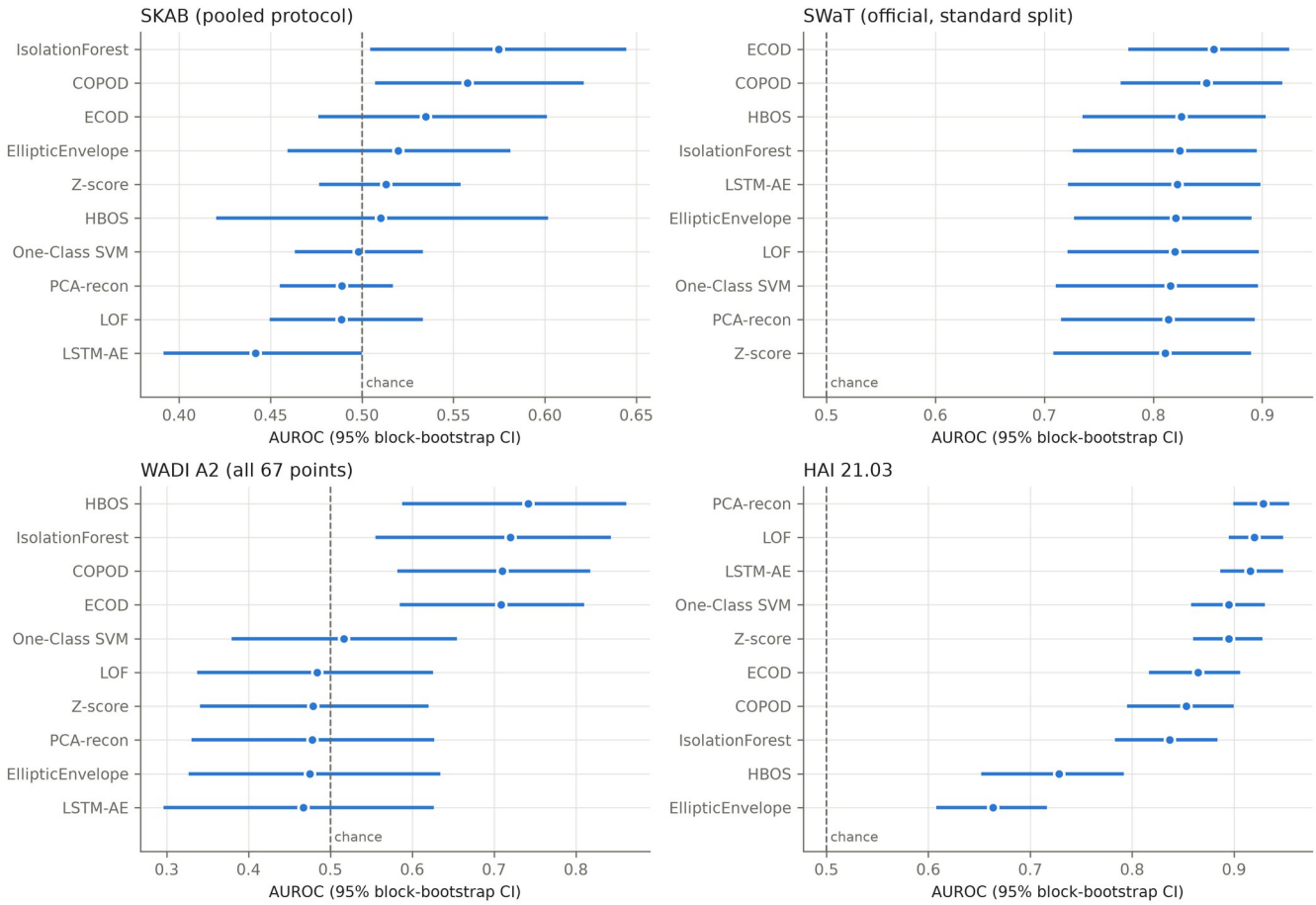


Figure 1. AUROC with 95% block-bootstrap confidence intervals (blocks: test files on SKAB, one-hour blocks on SWaT, WADI and HAI) for the seed-0 model; the intervals reflect test-block variability only. Dashed line: chance. On SKAB (pooled) all detectors are close to chance and only Isolation Forest's and COPOD's intervals exclude 0.5, narrowly; on SWaT (official release) all ten are well above chance with overlapping intervals; on WADI only HBOS, Isolation Forest, COPOD and ECOD are above chance while three analysers drift; on HAI a leading group separates from HBOS and Elliptic Envelope.

Detector	AUROC [95% CI]	AUPRC	MCC LF [95% CI]	FAR LF	Event rec. LF	Delay LF (s)	MCC VM
ECOD	0.856 [0.777, 0.925]	0.694	0.336 [0.216, 0.425]	0.002	0.26	41	0.617
COPOD	0.849 [0.770, 0.918]	0.726	0.628 [0.488, 0.730]	0.001	0.17	111	0.788
HBOS	0.826 [0.735, 0.903]	0.553	0.463 [0.306, 0.584]	0.118	0.63	20.5	0.440
Isolation Forest	0.824 [0.726, 0.895]	0.563	0.539 [0.369, 0.675]	0.070	0.69	12.5	0.552
LSTM-AE	0.822 [0.722, 0.899]	0.731	0.130 [0.056, 0.200]	0.742	0.89	0	0.101
Elliptic Envelope	0.821 [0.727, 0.890]	0.509	0.005 [−0.031, 0.052]	0.015	0.74	66	0.401
LOF	0.820 [0.721, 0.897]	0.718	0.149 [0.090, 0.212]	0.748	1.00	0	0.742
One-Class SVM	0.816 [0.711, 0.896]	0.718	0.184 [0.106, 0.274]	0.581	0.91	0	0.822
PCA-recon	0.814 [0.715, 0.893]	0.722	0.131 [0.055, 0.208]	0.726	0.91	0	0.078
z-score	0.811 [0.708, 0.890]	0.719	0.215 [0.109, 0.331]	0.421	0.91	0	0.820

Table 3. SWaT, official release, standard protocol, 25 sensors (test: full attack period, 449,919 rows, 12.1% attacks, 35 contiguous attack segments). Seed-0 models. AUROC and label-free MCC with 95% block-bootstrap CIs (one-hour blocks; test-block variability only); VM MCC is a point estimate. LF: label-free threshold (99th percentile of held-out normal-week scores); FAR: false-alarm rate on the attack period's normal rows; VM: validation-MCC threshold calibrated on the first third, scored on the last two thirds.

Detector	AUROC [95% CI]	AUPRC	MCC LF [95% CI]	FAR LF	Event rec. LF	Delay LF (s)	MCC VM
PCA-recon	0.929 [0.899, 0.954]	0.641	0.624 [0.572, 0.670]	0.016	1.00	1.5	0.678
LOF	0.920 [0.895, 0.948]	0.617	0.626 [0.570, 0.675]	0.016	1.00	2	0.691
LSTM-AE	0.916 [0.887, 0.948]	0.585	0.336 [0.284, 0.391]	0.091	1.00	5	0.664
One-Class SVM	0.895 [0.858, 0.930]	0.590	0.627 [0.573, 0.677]	0.012	1.00	3	0.662
z-score	0.895 [0.860, 0.928]	0.539	0.597 [0.544, 0.648]	0.013	1.00	3	0.653
ECOD	0.865 [0.817, 0.906]	0.470	0.519 [0.446, 0.580]	0.007	0.78	24	0.515
COPOD	0.853 [0.795, 0.900]	0.479	0.531 [0.467, 0.591]	0.004	0.80	22	0.513
Isolation Forest	0.837 [0.783, 0.884]	0.359	0.414 [0.343, 0.484]	0.004	0.66	27	0.425
HBOS	0.728 [0.652, 0.792]	0.115	0.145 [0.078, 0.214]	0.011	0.42	36	0.121
Elliptic Envelope	0.663 [0.608, 0.716]	0.114	0.179 [0.119, 0.248]	0.007	0.48	38	0.177

Table 4. HAI 21.03 (test: five files, 402,005 rows, 2.2% attacks, 50 attacks). Seed-0 models. AUROC and label-free MCC with 95% block-bootstrap CIs (one-hour blocks; test-block variability only); VM MCC is a point estimate. FAR: false-alarm rate on normal test rows; VM: validation-MCC threshold calibrated on test file 1, scored on files 2–5.

	All 67 points			Without 3 drifting analysers		
Detector	AUROC [95% CI]	MCC LF	FAR LF	AUROC [95% CI]	MCC LF	FAR LF
HBOS	0.742 [0.588, 0.862]	0.070	0.014	0.748 [0.602, 0.864]	0.022	0.012
Isolation Forest	0.720 [0.555, 0.843]	0.031	0.013	0.720 [0.570, 0.835]	0.029	0.013
COPOD	0.710 [0.582, 0.818]	0.055	0.009	0.709 [0.576, 0.817]	0.054	0.009
ECOD	0.709 [0.585, 0.810]	0.061	0.009	0.703 [0.567, 0.808]	0.060	0.009
One-Class SVM	0.517 [0.380, 0.655]	−0.030	0.559	0.733 [0.590, 0.859]	0.183	0.010
LOF	0.484 [0.337, 0.625]	−0.026	0.558	0.649 [0.502, 0.762]	0.230	0.006
z-score	0.479 [0.341, 0.620]	−0.027	0.559	0.697 [0.574, 0.807]	0.214	0.009
PCA-recon	0.478 [0.330, 0.627]	−0.027	0.558	0.686 [0.479, 0.832]	0.191	0.010
Elliptic Envelope	0.475 [0.327, 0.635]	−0.074	0.559	0.709 [0.551, 0.844]	−0.006	0.011
LSTM-AE	0.467 [0.296, 0.627]	0.019	0.560	0.695 [0.471, 0.876]	0.329	0.011

Table 5. WADI A2 (test: 172,803 rows, 5.8% attacks, 14 attacks), with all 67 points and without the three drifting analysers (64 points). AUROC with 95% block-bootstrap CI (one-hour blocks); label-free MCC and false-alarm rate on normal test rows.

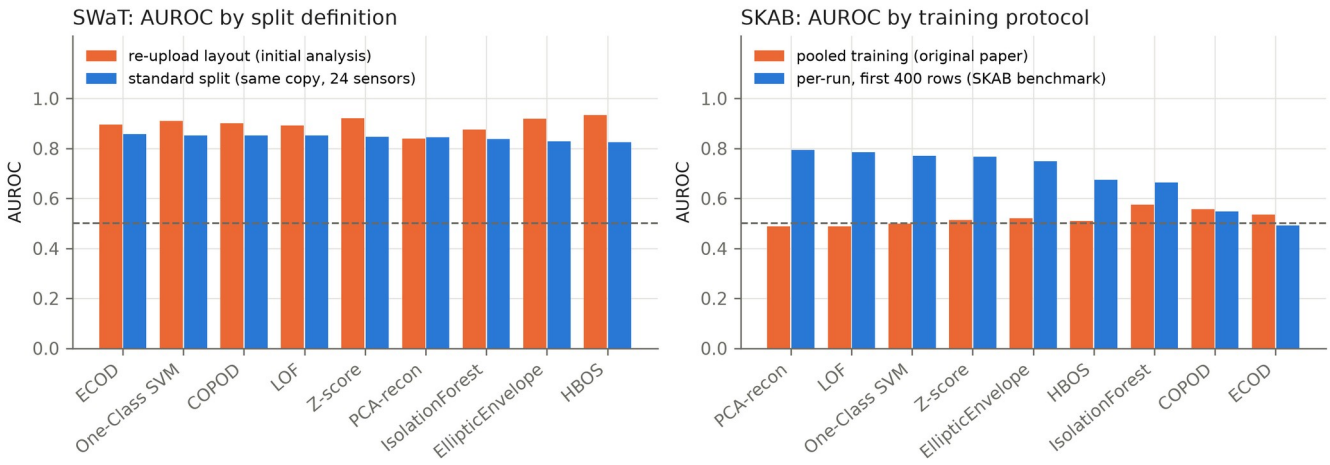


Figure 2. Same detectors, same code, different protocol. Left: SWaT AUROC under the re-upload layout used in the initial analysis versus the standard split. Right: SKAB AUROC with one model trained on all runs (pooled) versus one model per run trained on its first 400 rows (SKAB reference protocol; mean over runs).

Detector	F1	FAR	MAR	MCC	Mean run AUROC
Elliptic Envelope	0.644	0.202	0.445	0.361	0.749
z-score	0.722	0.468	0.213	0.331	0.766
LOF	0.723	0.556	0.168	0.302	0.785
One-Class SVM	0.732	0.712	0.077	0.277	0.771
PCA-recon	0.731	0.707	0.081	0.276	0.794
HBOS	0.603	0.306	0.457	0.239	0.674
Isolation Forest	0.470	0.201	0.641	0.174	0.664
COPOD	0.044	0.024	0.977	−0.004	0.548
ECOD	0.027	0.022	0.986	−0.029	0.493

Table 6. SKAB reference protocol (per run: first 400 rows train, remainder test; label-free threshold from each run's training scores). FAR and MAR as defined by the SKAB benchmark (false-alarm and missed-alarm rates).

	Selection	$P = 1$	$P = 2$	$P = 3$	$P = 4$	$P = 6$	$P = 12$	$P = 24$	$P = 48$	KS
IF	oracle	0.567 (239)	0.502 (120)	0.478 (80)	0.436 (60)	0.404 (40)	0.349 (20)	0.297 (10)	0.288 (5)	0.549 (89)
	pred.-normal	0.004 (95)	0.000 (12)	0.000 (14)	0.000 (10)	0.000 (7)	0.000 (3)	0.008 (2)	0.069 (3)	0.000 (24)
	all rows	0.206 (244)	0.149 (122)	0.255 (81)	0.084 (61)	0.138 (40)	0.292 (20)	0.221 (10)	0.153 (5)	0.188 (92)
EE	oracle	0.705 (239)	0.706 (120)	0.716 (80)	0.715 (60)	0.722 (40)	0.727 (20)	0.724 (10)	0.732 (5)	0.709 (89)
	pred.-normal	0.719 (239)	0.724 (120)	0.704 (79)	0.724 (60)	0.703 (39)	0.723 (20)	0.746 (10)	0.749 (5)	0.703 (89)
	all rows	0.329 (244)	0.405 (122)	0.420 (81)	0.530 (61)	0.422 (40)	0.718 (20)	0.697 (10)	0.678 (5)	0.334 (92)

Table 7. Retraining budget on SWaT (official release, 25 sensors): MCC over the attack period (point-wise) for each detector and retraining-data selection, at every periodic budget P (refit attempt every P hourly batches; 244 batches in total) and for the KS-triggered policy. The number in brackets is the number of successful refits. Single stream, so no confidence intervals. Static: Isolation Forest 0.165, Elliptic Envelope 0.724.

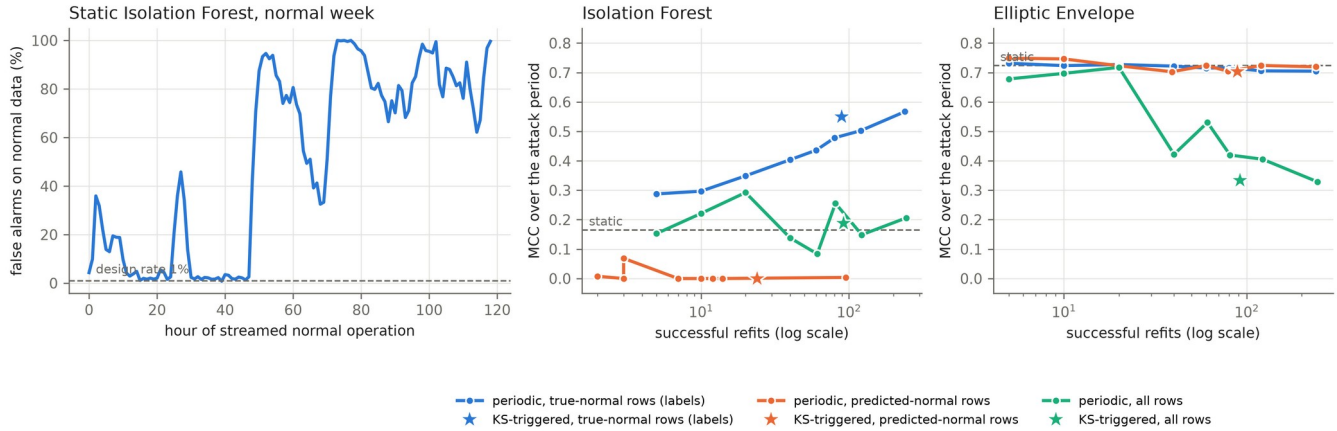


Figure 3. Left: false-alarm rate of a static Isolation Forest over SWaT's normal week (official release; threshold set to flag 1% of its training window). Middle and right: detection skill (MCC) over the attack period against the number of successful refits; lines are periodic retraining at different budgets, stars the KS-triggered policy, the dashed line the static model. Colours give the retraining-data selection.

Detector	Fit (s)	Load (ms)	p50 (ms)	p95 (ms)	p99 (ms)	Batch rows/s	Size (KB)	Peak RSS increase (MB)
z-score	0.006	0.12	0.002	0.002	0.002	7,188,644	2	107
HBOS	1.10	0.24	0.050	0.053	0.070	812,430	802	164
PCA-recon	0.019	0.18	0.058	0.063	0.072	1,242,616	52	71
Elliptic Envelope	4.22	0.31	0.151	0.161	0.187	5,866,393	503	89
One-Class SVM	3.10	2.78	0.158	0.172	0.232	10,049	1,942	228
LOF	2.55	1.88	0.892	0.934	0.978	7,745	22,188	128
Isolation Forest	0.072	7.46	2.255	2.353	2.431	853,367	1,149	85
ECOD	0.83	196.5	698.2	956.7	1,117.1	8,960	235,157	516
COPOD	1.11	222.6	739.3	833.9	880.7	11,203	235,157	527

Table 8. Cost per detector in a Docker container capped at one CPU and 1 GB RAM (linux/arm64 on an Apple M4 host; single BLAS thread; one spawned process per detector). Input: 50,000 normal training rows of the public re-upload's 24 SWaT sensors (120 features), not the 25-sensor official set (125 features) used elsewhere. Latency: one reading per call, 1,000 calls after warm-up (median, 95th and 99th percentiles). Peak RSS increase: peak resident memory above the measured baseline of about 444 MB, during fitting and scoring, read from the kernel's high-water mark reset after loading; it includes transient scoring buffers.

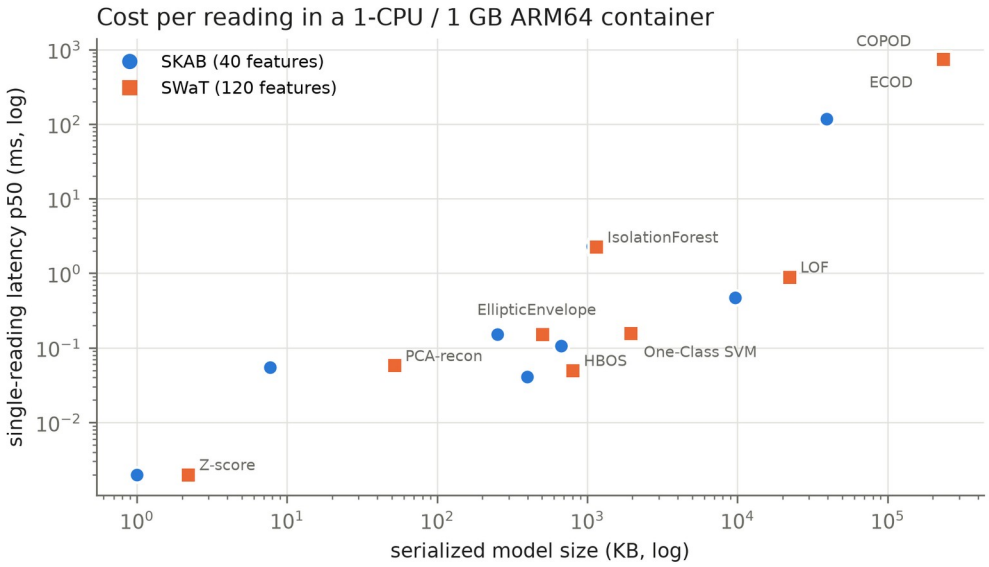


Figure 4. Single-reading latency against serialised model size (both log scale) in the capped container, for SKAB-shaped (40 features) and SWaT-shaped (120 features) input.

Choice	Evidence in this study	Implication
Sensor set	Adding one drifting SWaT analyser (AIT201): z-score false alarms 1.4% → 42%. Removing three drifting WADI analysers: false alarms 56% → 1% for six detectors	Check sensor ranges against training data; prefer detectors that bound each sensor's influence when analysers can drift
Training/test split	SWaT re-upload layout vs standard split: AUROC 0.84–0.93 vs 0.83–0.86; HBOS first vs last. SKAB pooled vs per-run: chance-level vs run AUROC up to 0.79, ranking inverted	State the time span and source of training and test data; use a benchmark's reference protocol
Threshold rule	Shared contamination vs per-model calibration: z-score MCC 0.18 vs 0.84. Label-free vs label-calibrated on SWaT: z-score MCC 0.22 vs 0.82, One-Class SVM 0.18 vs 0.82	Report a label-free threshold as the deployable result; treat label-calibrated results as an optimistic reference
Retraining data	Isolation Forest: oracle labels MCC 0.17 → 0.57; predicted-normal: collapse to flag-everything (MCC ≤ 0.07); all rows: attacks absorbed (MCC ≤ 0.30). Elliptic Envelope: predicted-normal MCC 0.70–0.75; all rows 0.33–0.53 with 40 or more refits	Do not claim adaptation gains from label-selected retraining; vet retraining data or use slowly changing detectors
Retraining trigger	KS trigger: 0.549 with 89 refits vs periodic 0.478 (80), 0.502 (120), 0.567 (239); KS checks 98 s vs every-batch refits 18 s	Compare triggers against a budget curve, including the cost of the check; one stream only
Cost measurement	Batch-averaged vs single-reading latency for Isolation Forest: 0.0019 vs 2.3 ms; feature update 3.2 ms	Report single-reading latency and memory above a stated baseline

Table 9. Summary: how each evaluation choice changed the measured outcome in this study, and the practical implication.